Multi-Instance Sync - Developer Guide

Hub-and-spoke, eventual consistency, offline-first (app-level CDC)

The ERP runs a full stack (Django + Postgres) at an online MASTER and one per AGENCY. Agencies operate offline and reconcile with the master. Sync is application-level Change-Data-Capture (an outbox), NOT database replication. Model: hub-and-spoke, master-authoritative for config, eventual consistency. All syncable tables use UUID PKs and agency-namespaced human numbers, so rows created anywhere are globally unique.

Components (apps/sync/)

- membership.py - deny-list deciding what syncs: is_syncable_model() / syncable_models(). Everything syncs except BLOCKLIST_APPS (admin, auth, contenttypes, sessions, token_blacklist, sync, sync_scig_sage, whatsapp_bot, online_booking, dead legacy) and BLOCKLIST_MODELS (core.auditlog, core.sequence, auth sessions). ~113 business models.
- models.ChangeLog - the outbox / CDC log (see fields below).
- capture.py - post_save/post_delete signals record local writes into ChangeLog; suspend_capture() disables it while applying remote changes; also suspended during migrate.
- apply.py::apply_change - idempotent apply of one entry.
- snapshot.py::full_snapshot - the bootstrap payload (all syncable rows).
- views.py - SyncSnapshotView, SyncPushView (+_relog), SyncPullView; IsSyncPeer + schema guard.
- schema.py::schema_fingerprint - sha256 of applied migrations (version parity).
- commands: bootstrap_from_master, sync_once (agency worker), reserve_worker.

The ChangeLog (outbox)

```
ChangeLog:
  entity       'app_label.model' (lowercased)
  object_id    UUID of the row
  op           'upsert' | 'delete'
  payload      full-row snapshot (FKs as UUID strings, JSON-safe)
  version      row.updated_at  -> drives last-write-wins
  origin_node  NODE_ID that produced it  -> fan-out & no-echo
  synced_at    NULL until pushed (agency out-tray marker)
  created_at   id order = replay order
```

Bootstrap (one-time, seeds a full replica)

1. GET /sync/snapshot/

Agency calls it (bootstrap_from_master). IsSyncPeer checks the SYNC_TOKEN bearer; X-Schema-Version must equal the master's fingerprint or 409.

2. full_snapshot()

Master emits every syncable row as upsert entries.

3. apply within one txn

Agency applies under transaction.atomic()+constraint_checks_disabled() (FKs deferred to COMMIT) and suspend_capture() (no echo). Order-independent.

4. set watermark

pull_cursor = snapshot cursor (master ChangeLog head).

Steady state (per sync_once cycle, ~90s)

1. Capture

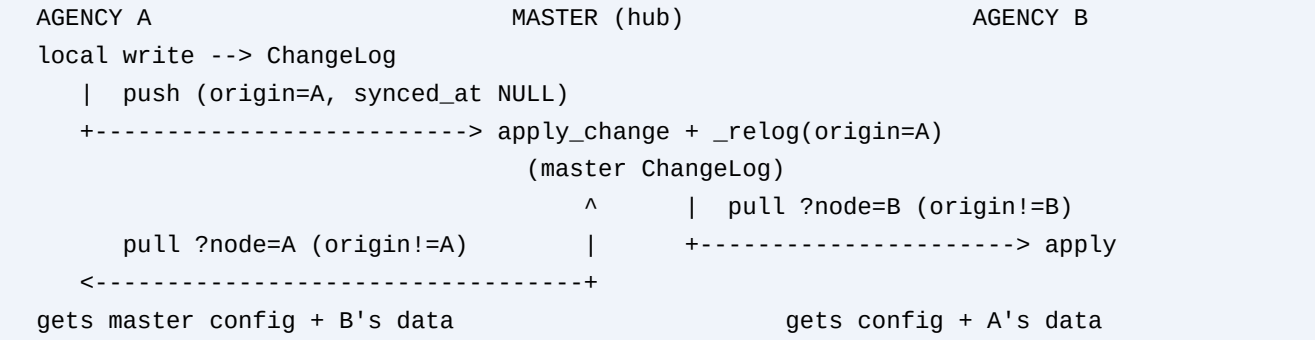
Any local write to a syncable model -> ChangeLog row, origin_node=NODE_ID, synced_at=NULL. Remote applies are under suspend_capture, so they are NOT captured.

2. Push

POST /sync/push/ with all local pending (synced_at IS NULL AND origin_node=me). Master apply_change() each; if 'applied', _relog() re-inserts it into the master ChangeLog PRESERVING origin_node (fan-out). Agency marks the batch synced_at=now.

3. Pull

GET /sync/pull/?since=<cursor>&node=<me>. Master reads a window id>since, cursor=max(window id) (monotonic, includes skipped), returns entries where origin_node != me. Agency apply_change() each (suspend_capture) and advances pull_cursor.



apply_change() semantics

- Lookup by PK. If present: LWW - skip when local.updated_at > incoming.version; else update via QuerySet.update (bypasses auto_now to preserve version).
- If absent: force_insert with the incoming PK. On IntegrityError (a locally-seeded row shares a unique natural key under a different PK), _unique_conflict() finds it, deletes it, re-inserts - master wins. This makes the full snapshot land without every ref table being deterministic.
- delete op -> soft-delete (is_deleted) tombstone if the model supports it, else hard delete.
- Runs inside suspend_capture() so applying never re-enters the ChangeLog (no echo/loop).

Consistency & conflicts

- Single-origin per row: each row is created by exactly one node (UUID) -> inserts never collide.
- Last-write-wins on edits, keyed by version (updated_at).
- Config is master-authored: a write-guard (guard.py) blocks agency writes to DOWN-registered config models; agencies freely create operational data. Config flows down, ops flow up+out.
- Deterministic UUIDs (admin_rbac.0023 AppModule/Permission via uuid5(code); core.0004 Currency) keep migration-seeded reference tables identical across instances (less churn).

Safety guards

- IsSyncPeer - shared SYNC_TOKEN bearer on every /sync/ endpoint (JWT-bypassed).

- schema_fingerprint - push/pull/snapshot return 409 on X-Schema-Version mismatch; both instances must run the same migrations. Fix a 409 by rebuilding both from the same commit.

- Capture suspended during migrate (pre_migrate/post_migrate) - seeds are baseline, not deltas, and sync_changelog may not exist yet on a fresh DB.

Config / ops

```
SYNC_ENABLED=1                  # capture + guards active
NODE_ROLE=master | agency
NODE_ID=<uuid>                  # agency: unique; used as origin_node
NODE_AGENCY_ID=<agency uuid>    # agency: must exist on master
SYNC_MASTER_URL=http://<host>:8080/api/v1   # agency -> master nginx
SYNC_TOKEN=<shared secret>      # identical on master + agencies
# workers (agency): sync_once loop (~90s), reserve_worker (~20s)
```

Known limits / TODO

- Not real-time: convergence lags by the worker interval; offline widens it (CAP - offline over strong consistency by design).

- Auto-created M2M through tables (non-UUID PKs) are excluded; m2m_changed capture is a TODO.

- Fan-out relays every agency's changes to every agency via the hub: more bandwidth + a shared ChangeLog that grows; prune/compaction is a future concern.

- Media files (uploads) are not synced by the ChangeLog - only DB rows.

Mental model: ChangeLog = per-node outbox + the master's shared inbox. Capture fills it, push drains it to the master, _relog fans it out, pull drains the master to peers, apply_change is the idempotent LWW writer, and suspend_capture breaks the echo loop.